

Lens: AI Codebase Explainer

Automated repository parsing, semantic codebase visualization, and intelligent context-aware RAG.

Overview

Lens is an advanced, AI-powered developer tool designed to automatically ingest, parse, and map public Python repositories. It provides developers with an intelligent **Semantic Code Search** and **Retrieval-Augmented Generation (RAG)** interface to ask complex architectural questions and get context-aware answers instantly.

By combining static syntax tree parsing with modern vector embeddings and Large Language Models (LLMs), Lens bridges the gap between raw source code and high-level architectural understanding.

Key Features

1. Automated Repository Ingestion

Simply paste a GitHub URL (for example, `encode/httpx`), and the system clones the repository, enforces resource limits, and maps the file structure.

2. Deep Abstract Syntax Tree (AST) Parsing

Lens uses **Tree-sitter** to structurally parse Python code. It intelligently breaks down source code into distinct, logical entities:

- Modules
- Classes and Inheritances
- Functions and Methods
- Imports and Dependencies
- Docstrings and Signatures

3. Semantic Vector Search (Local AI)

Extracted code entities are converted into dense vector embeddings using **Sentence-Transformers** (`all-MiniLM-L6-v2`). These embeddings are stored efficiently in a **SQLite Vector Store**, allowing for fast, offline, semantic similarity search without relying on exact keyword matches.

4. Context-Aware AI Answers (Gemini RAG)

When a user asks a question, the backend retrieves the most semantically relevant code chunks from SQLite. These chunks are injected into a prompt as highly specific context, and **Google Gemini** (`gemini-flash-latest`) generates an accurate, grounded explanation directly referencing the codebase.

5. Local-First & Resource Efficient

The entire pipeline (parsing, graph generation, embeddings, and retrieval) runs completely locally. The LLM is only invoked at the final step to synthesize the answer, ensuring minimal API costs and maximum privacy.

System Architecture

The application is built using a modern, decoupled stack:

Backend: FastAPI (Python)

Async Processing Pipeline: Manages a background queue to stream the parsing and embedding of repositories without blocking the API.

Storage: Uses `aiosqlite` for asynchronous local database management, storing both the entity relationship graph and the embedding vectors.

Models Layer: Integration with `sentence-transformers` for local embedding generation and `google-genai` for the LLM inference.

Frontend: React + Vite (TypeScript)

Dynamic UI: A polished, single-page application built with React and styled with modern CSS features (glassmorphism, CSS animations).

Real-time Progress: Polls the backend to show the exact status of the ingestion pipeline (Cloning → Parsing → Mapping → Indexing).

Chat Interface: Displays the AI-generated answer alongside the exact file paths and code snippets retrieved by the RAG system.

Pipeline Workflow

Step 1 Clone

`gitpython` executes a shallow clone of the target repository.

Step 2 Parse

The pipeline streams Python files one by one while Tree-sitter extracts AST nodes.

Step 3 Map

An in-memory graph connects modules, imports, and classes, which is subsequently saved to SQLite.

Step 4 Store

Text chunks (combining signatures, docstrings, and raw code) are converted to NumPy arrays via `MiniLM-L6-v2` and saved as blob data in SQLite.

Step 5 Ready

The repository becomes fully compiled and available in the user interface.

Step 6 Query

The user asks a question. The query is embedded, cosine similarity is computed against the SQLite store, and Gemini generates a final response based on the top-K chunks.